THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Patiala, Punjab

PROJECT TEAM MATCHER

An Intelligent Team Formation Platform

Progression Report and Implementation Plan

Course:	UCS503P
Submitted to:	Kritika Goel
Department:	Computer Science and Engineering
Institute:	TIET, Patiala
Project Type:	Software Engineering Project
Development Period:	August–October 2026
Platform:	Web Application

Project Team

Mohit Garg	1024031006
Paras Sharma	1024030609
Manan Dhingra	1024030222
Parv Panwar	1024030226

October 2026

Contents

1 Executive Summary **3**

2 Current Project Status **4**
2.1 Development Progression 4

3 Work Completed So Far **5**
3.1 Core Bug Fixes and Architecture Stabilisation 5
3.2 Page Wiring and Form Fixes 5
3.3 UI and Design-System Foundation 6

4 Software Development Life Cycle (SDLC) **7**
4.1 Overview 7
4.2 Phase 1: Planning and Feasibility 7
4.3 Phase 2: Requirements Analysis 8
4.4 Phase 3: System Design 8
4.5 Phase 4: Implementation 8
4.6 Phase 5: Testing 9
4.7 Phase 6: Deployment 9
4.8 Phase 7: Maintenance and Future Enhancement 9
4.9 SDLC Status Summary 10

5 Google OAuth Authentication and Access Control **10**
5.1 Purpose and Current Status 10
5.2 Planned Requirements 10
5.3 Planned Authentication Flow 11
5.4 Eligibility Verification Considerations 11

6 Technical Challenges and Problems Encountered **12**
6.1 Problem 1: Overlapping Hackathon Listings 12
6.2 Problem 2: Race Conditions During Team Joining 13

7 Risk and Mitigation **15**

8 Implementation Plan **16**
8.1 Revised Roadmap and Target Dates 16
8.2 Stage 1: HTML Page UI Migration 17
8.3 Stage 2: Secondary CSS Modernisation and Responsive Design 17
8.4 Stage 3: Dynamic Interaction and Animation 18

8.5 Stage 4: Data-Integrity Testing for Hackathon Listings 18
8.6 Stage 5: Backend and Database Integration 18
8.7 Stage 6: Google OAuth Integration 18
8.8 Stage 7: Testing and Verification 19
8.9 Stage 8: Deployment and Final Documentation 19

9 Milestone Completion Summary **20**

10 Current Limitations and Dependencies **20**

11 Conclusion and Next Steps **21**

1. Executive Summary

Project Team Matcher, currently branded in the implementation as *TeamForge*, is being developed as a Thapar-focused platform through which students can find project partners, assemble hackathon teams, and review join requests. The current implementation operates as a standalone frontend simulation using browser `localStorage`; integration with the Node.js and MySQL backend has not yet been completed.

The initial project audit and core functional repair work are complete. Broken JavaScript syntax and incomplete backend fetch calls were addressed by introducing a unified local data-access layer. The global styling foundation has also been updated with a Thapar crimson-and-white palette, modern typography, glassmorphism, reusable cards, and animation styles. The current priority is migrating the remaining HTML pages to the new design system, followed by responsive checks, interaction polish, end-to-end verification, and backend integration.

This revision of the report additionally:

- maps the project to the standard **Software Development Life Cycle (SDLC)** and records the current status of each phase (Section 4);
- describes the **planned** Google OAuth 2.0 / OpenID Connect sign-in and the access-control rules that will keep the platform restricted to eligible Thapar students (Section 5);
- documents two technical challenges identified during development, namely **overlapping hackathon listings** and **race conditions during team joining**, together with proposed mitigations (Section 6);
- adds a risk and mitigation register and revises the implementation roadmap accordingly (Sections 7 and 8).

Google OAuth, concurrency-safe joining, the MySQL backend, and deployment are *not* implemented at the time of writing; they are presented throughout as planned or proposed work.

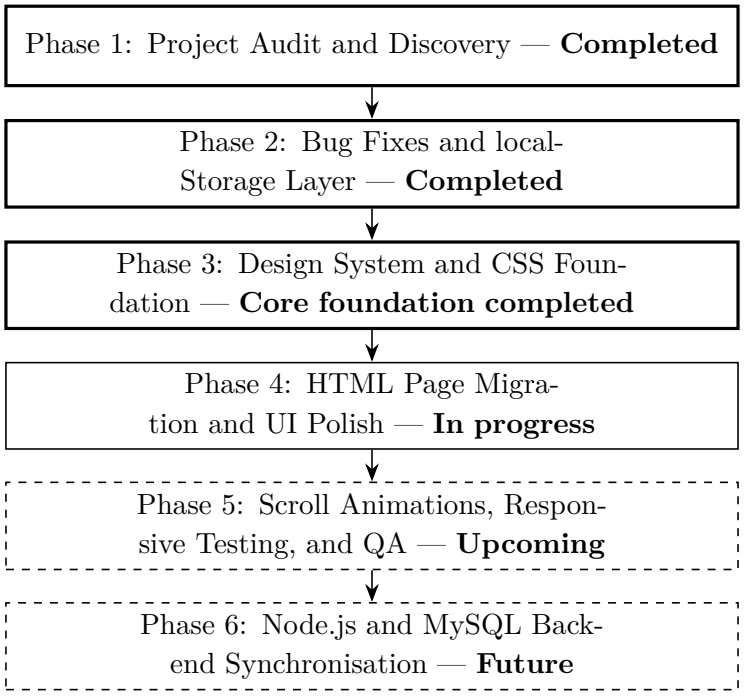
2. Current Project Status

Table 1: Summary of current project status.

Project status	Phase 2 complete; Phase 3/UI migration work in progress
Architecture mode	Standalone `localStorage` simulation; pre-backend integration
Completed focus	Project audit, JavaScript bug fixes, `localStorage` layer, core CSS design system
Immediate focus	HTML page migration and visual consistency
Next major milestones	Responsive styling, dynamic interaction polish, end-to-end QA, backend/database wiring
Planned additions	Google OAuth sign-in with institutional-domain validation; concurrency-safe team joining; hackathon-listing data integrity

2.1 Development Progression

Figure 1 shows the implementation phases used by the team. These implementation phases are distinct from, and nested within, the SDLC phases described in Section 4.



Border style: **thick** = completed; thin = in progress; dashed = upcoming/future.

Figure 1: High-level progression of the implementation.

3. Work Completed So Far

3.1 Core Bug Fixes and Architecture Stabilisation

The frontend previously contained an incomplete transition between browser-based storage and unfinished Express backend endpoints. The following changes were reported as completed.

Table 2: Core bug fixes and architecture stabilisation.

Component	Issue identified	Resolution applied	Status
`js/main.js`	Syntax errors, duplicate storage methods, and broken fetch calls.	Refactored into a unified API-style client that performs `localStorage` operations and includes error handling.	Completed
`js/auth.js`	Backend fetch calls caused login and registration to fail silently.	Switched to client-side authentication against mock/local storage and added input validation.	Completed
`js/mock-data.js`	Dataset contained only one student and two teams, with missing IDs and counters.	Expanded the mock dataset to three students and four hackathon teams, with skill tags, avatars, and incrementing counters.	Completed
`js/dashboard.js`	Dashboard selected arbitrary teams instead of the logged-in student's teams.	Updated the logic for joined teams, pending join counts, and personalised statistics.	Completed
`js/teams.js`	Search did not cover descriptions and skills; empty states were missing.	Added multi-field search across titles, descriptions, and required skills, with empty-state notifications.	Completed
`js/team-details.js`	No leader controls, duplicate requests were possible, and capacity was not enforced in the interface.	Added Accept/Reject moderation controls, duplicate-request guards, and an automatic full-status lock (interface level, within the local simulation).	Completed
`js/profile.js`	Null references for phone, GitHub, and LinkedIn fields were not handled.	Added safe fallback handling and formatted social-link presentation.	Completed

3.2 Page Wiring and Form Fixes

Table 3: Page wiring and form fixes.

Page	Work completed	Status
`pages/login.html`	Added the missing `mock-data.js` script dependency so the initial login state can initialise correctly.	Completed
`pages/register.html`	Added explicit input IDs (`regName`, `regStudentId`, `regEmail`, `regGithub`, and `regSkills`) and connected them to `auth.js`.	Completed
`pages/create-team.html`	Connected form submission to `api.createTeam()`, including auto-incrementing team IDs and redirects.	Completed
`pages/edit-profile.html`	Added input bindings for profile fields such as bio, links, and skills, with `localStorage` persistence.	Completed
`pages/my-requests.html`	Updated request cards to show readable team names and hackathon information instead of numerical IDs.	Completed

3.3 UI and Design-System Foundation

Table 4: UI and design-system enhancements.

Area	Enhancements implemented	Status
Typography and palette	Integrated Google Fonts (Inter), Thapar crimson colours (`#B91C1C` primary, `#991B1B` dark red, and `#FEF2F2` soft tint), and dark-slate text.	Core styles done
Navigation bar	Added a sticky glassmorphic navbar with backdrop blur, a subtle bottom border, hover highlights, and a primary call-to-action button.	Done
Component cards	Added reusable styles for feature, step, team, action, and statistic cards, including hover lift and shadows.	Done
Animations	Added `fadeUp` and `shimmer` keyframes, reveal-state classes, and button active interactions.	CSS done

The core design foundation is implemented in `css/style.css`. Authentication-specific and responsive styling, as well as final page-by-page migration, remain part of the implementation plan.

4. Software Development Life Cycle (SDLC)

4.1 Overview

The Software Development Life Cycle (SDLC) is a structured sequence of phases through which a software product is planned, specified, designed, built, tested, released, and maintained. Following the standard SDLC gives the team a shared vocabulary for progress, makes dependencies between activities explicit (for example, concurrency testing cannot be completed before the backend exists), and ensures that requirements documented in the SRS are traced through to design, implementation, and testing.

Project Team Matcher follows the phases shown in Figure 2. Development is iterative: findings from testing and maintenance feed back into planning and requirements for the next iteration.

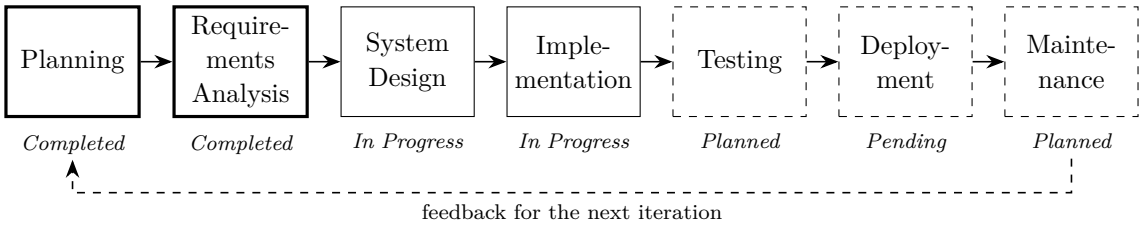


Figure 2: Standard SDLC applied to Project Team Matcher, with the current status of each phase (thick border = completed; thin = in progress; dashed = not yet started).

4.2 Phase 1: Planning and Feasibility

- **Problem identification:** Thapar students frequently struggle to find teammates with complementary skills for course projects and hackathons; team formation currently depends on informal contacts and group chats.
- **Objectives and scope:** provide a single platform for student profiles, skill-based team discovery, team creation, and moderated join requests, limited to eligible Thapar students.
- **Stakeholders:** students (team seekers and team leaders), the project team, and the course instructor as evaluator.
- **Technical feasibility:** the chosen stack (HTML5, CSS3, and Vanilla JavaScript on the frontend; Node.js and Express.js on the backend; MySQL for persistence) is widely documented, free to use, and within the team's skills, making it feasible for the August–October 2026 development period.

Current status: **Completed (project proposal)**

4.3 Phase 2: Requirements Analysis

- Define functional requirements: student authentication, profile management, team creation, team discovery, search and filtering, join requests and leader moderation, and privacy of completed teams.
- Define non-functional requirements: security of student data, usability, responsiveness across devices, reliability of team-capacity rules, and maintainability.
- The existing **Software Requirements Specification (SRS)** is the primary requirements reference for this report.

Current status: **Completed (SRS baseline); OAuth and concurrency requirements in this report to be reflected in the SRS**

4.4 Phase 3: System Design

- **Architecture:** a client–server architecture in which the browser frontend communicates with an Express.js REST API, which in turn is the only component permitted to access the MySQL database.
- **Component design:** frontend pages and the shared design system; backend API routes; database schema; and the authentication flow (Section 5).
- **Data design:** entities and relationships for *students*, *profiles*, *skills*, *teams*, *memberships*, *hackathons*, and *join requests*. A student may lead or belong to many teams; each team belongs to at most one hackathon; memberships and join requests link students to teams.

Current status: **In Progress — frontend design system completed; database schema and authentication flow to be finalised**

4.5 Phase 4: Implementation

- Develop the HTML5, CSS3, and Vanilla JavaScript frontend (in progress; see Section 9).
- Develop the Node.js and Express.js backend and REST APIs.
- Implement MySQL persistence for all entities.
- Integrate Google OAuth when the planned authentication work begins.

It is important to distinguish two implementation states. The **current** version is a frontend simulation: all data are stored in each browser's `localStorage`, so there is no shared state between users and no server-side enforcement. The **future** version will replace this simulation with persistent backend APIs and a central MySQL database.

Current status: **In Progress (frontend simulation); backend implementation Pending**

4.6 Phase 5: Testing

Planned test areas include:

- registration and authentication, including rejected domains and cancelled sign-in attempts;
- profile management, team creation, searching, filtering, and join requests;
- overlapping or duplicate hackathon listings (Section 6.1);
- race conditions involving simultaneous join requests (Section 6.2);
- team-capacity enforcement, duplicate requests, authorisation checks, and privacy of completed teams.

Current status: **Planned — full-flow verification is scheduled after the UI work**

4.7 Phase 6: Deployment

The planned deployment process is:

1. **Environment configuration:** separate development and production settings supplied through environment variables.
2. **Secure credentials:** the Google OAuth client secret, session secret, and database password are kept out of source control and stored only in the hosting environment.
3. **Database configuration:** provision the production MySQL database, apply the schema, and use a database account with only the privileges the application requires.
4. **Deployment verification:** after release, run smoke tests for sign-in, team creation, and join requests, and confirm that HTTPS and secure cookies are active.

Current status: **Pending — not yet deployed; hosting platform to be confirmed**

4.8 Phase 7: Maintenance and Future Enhancement

- Fix defects reported during testing and evaluation, prioritised by severity, and re-test affected flows before each release.
- Apply security improvements such as dependency updates, rate limiting on authentication and join endpoints, and periodic review of access-control rules.
- Optimise performance (database indexes on frequently filtered columns, query tuning, and page-load improvements) and refine the database schema.
- Develop future features, for example skill-based teammate recommendations and notifications.

Current status: **Planned**

4.9 SDLC Status Summary

Table 5: Mapping of SDLC phases to project activities and current status.

SDLC Phase	Project Activities	Current Status
Planning and Feasibility	Problem definition, objectives, scope, stakeholders, and technology-stack feasibility (project proposal).	Completed
Requirements Analysis	Functional and non-functional requirements documented in the SRS.	Completed (SRS update for OAuth pending)
System Design	Client–server architecture and frontend design system completed; MySQL schema, API design, and OAuth flow to be finalised.	In Progress
Implementation	Frontend audit, bug fixes, `localStorage` layer, and CSS foundation completed; page migration ongoing; backend, MySQL, and OAuth not yet started.	In Progress
Testing	End-to-end, data-integrity, security, integration, and concurrency testing.	Planned
Deployment	Environment configuration, credential management, database provisioning, and release verification.	Pending
Maintenance and Enhancement	Defect fixing, security hardening, performance tuning, and new features.	Planned

5. Google OAuth Authentication and Access Control

5.1 Purpose and Current Status

The project intends to introduce **Google authentication** so that students can register and sign in without creating a separate password for the platform. This reduces registration effort and removes the need for the platform to store passwords.

Status: Planned. Google OAuth has *not* been implemented. The current version uses client-side authentication against mock data stored in `localStorage`, which is a development simulation only. Google sign-in will be implemented together with the Express.js backend, because the security checks described below must run on the server.

5.2 Planned Requirements

1. **Google sign-in:** students will be able to authenticate using their Google accounts.

2. **Trusted identity flow:** the system will validate the user's identity through the Google OAuth 2.0 / OpenID Connect authorisation-code flow. The backend will verify the ID token's signature, issuer, audience (the project's client ID), and expiry, and will use a `state` parameter to protect against cross-site request forgery.
3. **Restricted access:** access to the platform must remain restricted to eligible Thapar Institute students.
4. **Institutional domain validation:** where an institutional email restriction is adopted, the authenticated email will be validated against the approved domain (`@thapar.edu`), and only addresses that Google reports as verified will be accepted.
5. **Backend verification:** the email address and authentication claims will be verified on the backend; values submitted by the frontend (such as an email field or a client-side "logged-in" flag) will never be trusted.
6. **Secure sessions:** after successful verification the server will create a secure session (for example an `HttpOnly`, `Secure`, `SameSite` cookie) or issue a token that is validated on the server for every protected request. Tokens will not be stored in `localStorage`.
7. **Backend authorisation:** private student profiles, team information, and team-management operations (accepting or rejecting requests, editing or closing a team) will be protected by server-side authorisation checks, such as confirming that only the team leader can moderate that team's requests.
8. **Graceful failure handling:** authentication failures, cancelled login attempts, invalid or expired credentials, and unauthorised domains will return the user to the login page with a clear message; no account or session will be created in these cases.

5.3 Planned Authentication Flow

Figure 3 summarises the planned sign-in sequence, including the points at which a failed, cancelled, or unauthorised sign-in is rejected without creating a session.

5.4 Eligibility Verification Considerations

A Google account on its own is **not** proof that a user is an eligible Thapar student, because anyone can create a personal Google account. Eligibility can only be claimed when an approved verification mechanism is validated on the server, such as the institutional email domain. Even then, the team should confirm with the institute whether `@thapar.edu` accounts are issued only to current students or also to staff and alumni; if they are not student-only, an additional check (for example, matching the roll number pattern or an administrator-approved list) may be required.

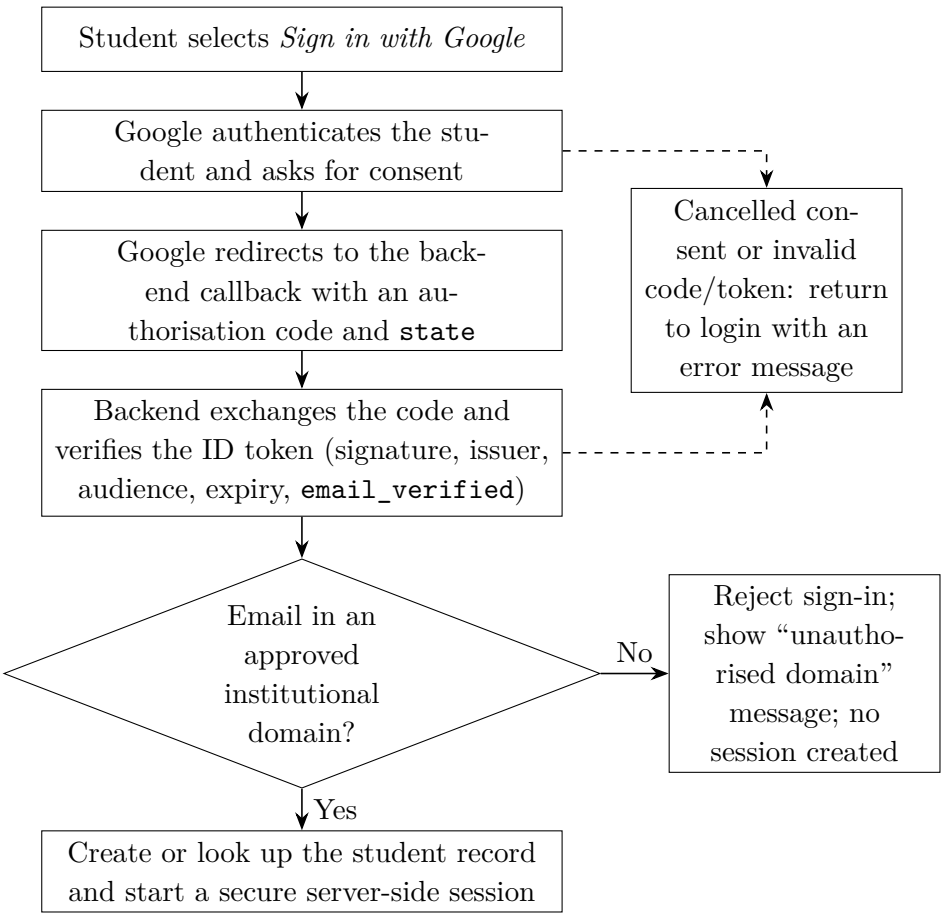


Figure 3: Planned Google OAuth 2.0 / OpenID Connect sign-in flow (not yet implemented). Dashed arrows indicate failure paths.

6. Technical Challenges and Problems Encountered

This section documents two problems identified during development. For each problem, the *observed issue* is described separately from the *proposed fixes*, none of which have been implemented yet.

6.1 Problem 1: Overlapping Hackathon Listings

Observed issue. Hackathon and project listings in the team-discovery interface can overlap, producing entries that appear to be duplicates or that present the same opportunity inconsistently. A student browsing teams may see what looks like the same hackathon several times under slightly different names, or two near-identical team cards, and cannot easily tell which listing is genuine or current. This reduces trust in the listings, splits interested students across duplicate entries, and can leave genuine teams with fewer join requests.

Contributing factors.

- **Inconsistent event names:** the same event may be entered with different spellings,

capitalisation, or year suffixes (for example, “HackThapar”, “Hack Thapar 2026”, and “hackthapar”), so name-based grouping treats them as separate events.

- **Duplicate records:** repeated form submissions or a leader re-creating a team can save the same team twice.
- **Poorly coordinated listing data:** without a single authoritative list of hackathons, each team defines its own event details, and those details can disagree (dates, organiser, mode).

Why identifiers and structure matter. Reliable listings require a *hackathon* to be a structured database record with a stable identifier, and each team to reference exactly one such record. The association rule needs to be defined explicitly: a team belongs to at most one hackathon, a hackathon can have many teams, and the team’s hackathon is selected from existing records rather than typed as free text. Filtering, validation, and deduplication can then operate on identifiers instead of display names.

Separate teams versus accidental duplicates. Several distinct teams taking part in the same hackathon is valid and expected, and must not be merged or hidden. Deduplication should therefore target only *accidental* duplicates, for example two teams with the same leader, the same hackathon identifier, and the same title, or a second hackathon record whose normalised name and dates match an existing one.

Proposed solutions (not yet implemented).

- Use stable hackathon identifiers instead of relying exclusively on display names.
- Validate event and team data before saving it: trim and normalise names, require valid dates, and warn when a near-identical record already exists.
- Apply appropriate database constraints where applicable, such as a foreign key from `teams.hackathon_id` to `hackathons.id` and a unique key on the normalised hackathon name and start date.
- Ensure the frontend filters and groups teams by consistent event identifiers rather than by name strings.
- Test the interface with multiple teams belonging to the same hackathon, and with intentionally duplicated records, to confirm that genuine teams are shown and accidental duplicates are prevented.

Current status. Identified; mitigations are proposed and scheduled for the data-integrity testing and backend stages of the roadmap (Section 8).

6.2 Problem 2: Race Conditions During Team Joining

Observed issue. A race condition can occur when multiple students attempt to take the last remaining position in a team at nearly the same moment.

Example scenario. A team requires five members and currently has four. Two students submit join requests at almost the same time. If both requests check the available

capacity *before* either request updates the database, both checks see four members and both succeed. Both students might then be accepted, leaving the team with six members and exceeding its maximum capacity. The same problem arises whether the membership is created automatically or when a leader's acceptance of two pending requests is processed concurrently (for example, from two browser tabs).

Why frontend validation alone is insufficient.

- Each browser holds its own, possibly outdated, copy of the team's state, so two clients can both believe that a position is available.
- The check and the update are separate steps; any request arriving between them sees the old member count.
- Client-side code can be bypassed by sending requests directly to the API.
- In the current `localStorage` simulation, the full-status lock in `team-details.js` only prevents over-filling within a single browser; it cannot coordinate between different users.

Capacity must therefore be enforced by the backend and the database, which are the only components shared by all users.

Proposed mitigations (not yet implemented).

1. Perform authoritative capacity validation on the backend for every join or acceptance request.
2. Use a database transaction so that checking capacity, inserting the membership, and updating the team's status succeed or fail together (atomically).
3. Use row-level locking (for example, `SELECT … FOR UPDATE` on the team row in MySQL InnoDB) or an equivalent concurrency-control mechanism, such as a conditional update that increments the member count only while it is below capacity.
4. Enforce database constraints, such as a unique key on (`team_id`, `student_id`) in the memberships table.
5. Check whether the join request is a duplicate or whether the student is already a member before creating a new request or membership.
6. Update the team's status to *completed/private* in the same transaction when the required capacity is reached.
7. Reject, or safely handle, additional requests once the team is full, returning a clear "team is full" message and marking any remaining pending requests accordingly.

Current status. Identified. The local simulation includes interface-level duplicate-request guards and a full-status lock (Table 2), but server-side capacity enforcement, transactions, locking, and database constraints have not been implemented and cannot be until the backend is connected to MySQL.

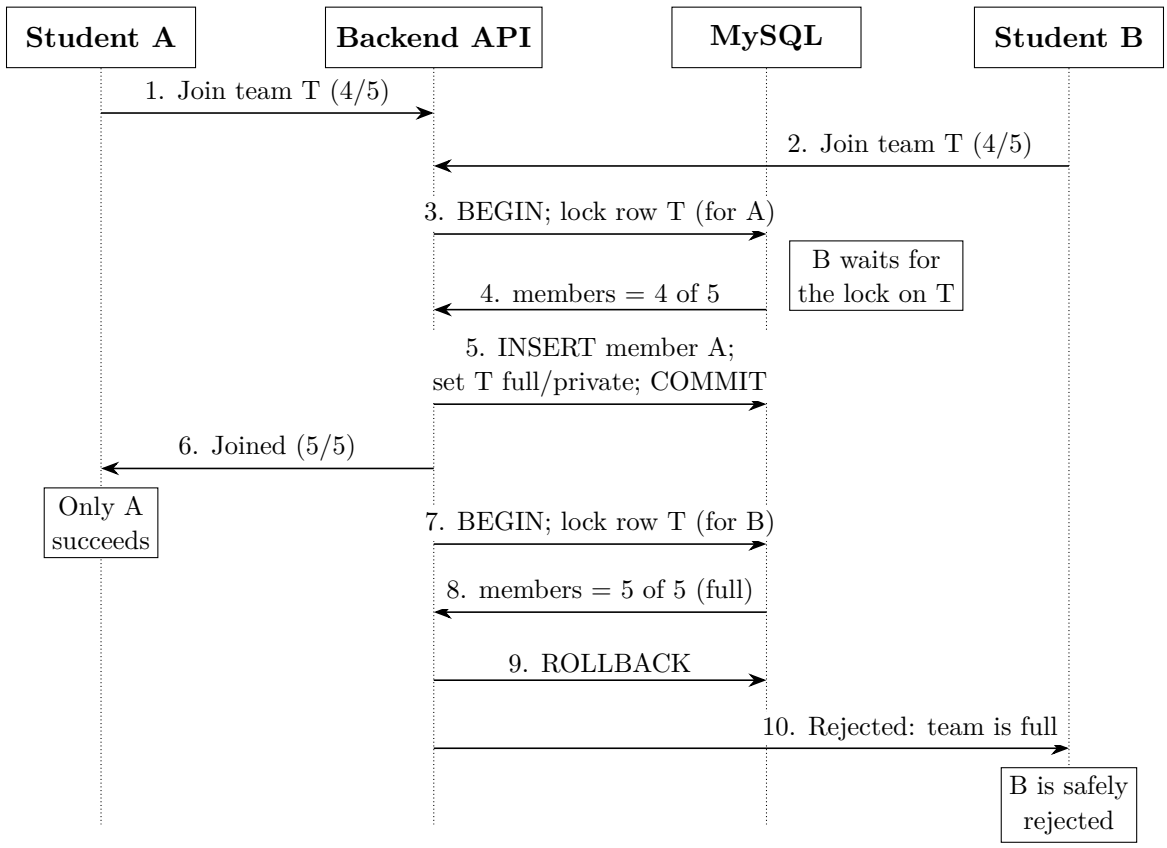


Figure 4: Proposed concurrency-safe handling of two simultaneous requests for the final position in team T. The backend serialises the requests with a transaction and a row lock, so only one membership operation succeeds. This is a design proposal and has not been implemented.

7. Risk and Mitigation

Table 6: Risk and mitigation register.

Issue	Impact	Proposed Mitigation	Status
Overlapping or duplicate hackathon listings	Confusing discovery; students split across duplicate entries; inconsistent event details.	Stable hackathon IDs; foreign key from teams to hackathons; name normalisation and validation; ID-based filtering; tests with many teams per hackathon.	Open — mitigation proposed
Concurrent join requests (race conditions)	Team exceeds maximum capacity; incorrect completed/private status.	Backend capacity check inside a transaction with row-level locking or a conditional update; status updated atomically.	Open — requires backend

Table 6 continued

Issue	Impact	Proposed Mitigation	Status
Unauthorised access (authentication or authorisation weaknesses)	Non-eligible users join; private profiles exposed; teams modified by non-leaders.	Planned Google OAuth with server-side ID-token and domain verification; secure sessions; per-route authorisation checks.	Open — OAuth planned
Inconsistent frontend and backend data	Pages show stale or conflicting information; actions fail after backend migration.	Treat the backend as the single source of truth; replace `localStorage` with API calls; consistent response formats and error handling; integration tests.	Open — migration pending
Duplicate join requests	Leader receives repeated requests; risk of duplicate membership.	Server-side check for existing pending request or membership; unique constraints on (team, student).	Partially mitigated (interface guard only)

8. Implementation Plan

The remaining work is organised into the following stages. Dates for the original tasks are the targets listed in the earlier implementation plan and may shift as development proceeds. Dates for newly added tasks are marked *To be confirmed* (TBC) until the team agrees a schedule.

8.1 Revised Roadmap and Target Dates

Table 7: Revised implementation roadmap.

No.	Activity	Target dates	Status
–	Project audit, bug fixes, and `localStorage` layer	10–11 Oct 2026	Completed
–	Design tokens and global CSS foundation	11–12 Oct 2026	Completed
1	Complete the remaining frontend (HTML page) migration	13–14 Oct 2026	In Progress
2	Improve responsive design (authentication and responsive CSS overhaul)	14–15 Oct 2026	In Progress
3	Implement and test dynamic UI interactions (scroll animation, navbar state, toasts)	15 Oct 2026	Planned

Table 7 continued

No.	Activity	Target dates	Status
4	Test overlapping hackathon listings and duplicate data	16–17 Oct 2026 (with full-flow verification)	Planned
5	Design and implement concurrency-safe team joining	TBC (requires item 7)	Planned
6	Integrate Google OAuth authentication with domain validation	TBC (requires item 7)	Planned
7	Connect the Node.js/Express.js backend to MySQL	18–21 Oct 2026	Planned
8	Replace `localStorage` simulation with persistent backend APIs	From 18 Oct 2026; completion TBC	Planned
9	End-to-end, security, integration, and concurrency testing	TBC	Pending
10	Deployment and final documentation	TBC	Pending

8.2 Stage 1: HTML Page UI Migration

- **Landing page (`index.html`):** unify branding as "TeamForge | Thapar Institute"; update the hero section, primary and outline buttons, "How It Works" cards, feature grid, and footer.
- **Dashboard (`pages/dashboard.html`):** standardise navigation, add a welcome banner, update Create Team and Browse Teams action cards, and format statistics cards.
- **Team pages:** update `teams.html` with a filter bar and team-card grid; improve `team-details.html` with leader approval cards, status badges, and member lists; style `create-team.html` forms.
- **User and profile pages:** update `profile.html`, create a two-column layout for `edit-profile.html`, and add clear Pending, Accepted, and Rejected status chips to `my-requests.html`.

8.3 Stage 2: Secondary CSS Modernisation and Responsive Design

- Update `css/auth.css` with a modern authentication card, crimson header strip, branded header, and improved form inputs.
- Update `css/responsive.css` with a mobile navigation drawer and responsive grids: one column below 768 px, two columns below 1024 px, and three columns on desktop.

- Harmonise or consolidate redundant rules in `dashboard.css`, `teams.css`, and `profile.css` to reduce style conflicts with `style.css`.

8.4 Stage 3: Dynamic Interaction and Animation

- Add a native `IntersectionObserver` in `main.js` to activate reveal animations as elements enter the viewport.
- Add a scroll listener to toggle the navbar's scrolled state, backdrop blur, and elevation.
- Replace browser `alert()` pop-ups with animated top-right toast notifications, using distinct success and error treatments.
- Test each interaction on desktop and mobile widths.

8.5 Stage 4: Data-Integrity Testing for Hackathon Listings

- Add mock data containing several genuine teams for the same hackathon and intentionally duplicated or inconsistently named events.
- Verify that search and filters group teams correctly and that duplicates are detected (Section 6.1).
- Record any defects found so that identifier-based fixes can be carried into the database schema.

8.6 Stage 5: Backend and Database Integration

- Implement Express server routes and connect them to MySQL.
- Create the schema for students, profiles, skills, teams, memberships, hackathons, and join requests, including foreign keys and unique constraints.
- Replace the `localStorage` data-access layer in `main.js` with calls to persistent backend APIs, keeping the same function interface where possible.
- Implement concurrency-safe team joining using transactions and row-level locking or conditional updates (Section 6.2).
- Replace mock-token/client-side authentication with session-based or JWT authentication validated on the server.

8.7 Stage 6: Google OAuth Integration

- Register the application with Google and configure the OAuth client and redirect URIs for development and production.
- Implement the authorisation-code flow, server-side ID-token verification, and institutional-domain validation (Section 5).
- Link verified Google identities to student records and protect all private routes with server-side authorisation.

8.8 Stage 7: Testing and Verification

The planned end-to-end verification sequence is:

1. Load the application with mock data (the source plan identifies the test user as Mohit Garg, ID 102024000).
2. Register a new student.
3. Create a team and verify that the dashboard updates.
4. Search teams by skill tag.
5. Send a join request and check the leader's approval interface.
6. Edit the profile and verify that the updated details persist.

The UI should also be checked for consistent TeamForge branding and the Thapar red-and-white theme across all eight pages. After backend integration, testing will be extended to:

- **Integration testing** of every API route against MySQL;
- **Security testing** of sign-in, rejected domains, cancelled logins, and attempts to access or modify other students' data;
- **Concurrency testing** by sending simultaneous join requests for the last position and confirming that exactly one succeeds;
- **Regression testing** of duplicate requests, capacity enforcement, and privacy of completed teams.

8.9 Stage 8: Deployment and Final Documentation

- Configure the production environment, secrets, and database as described in the Deployment phase of the SDLC (Section 4).
- Deploy, run post-deployment smoke tests, and record the results.
- Update the SRS, this report, and user documentation to reflect the final implementation.

9. Milestone Completion Summary

Table 8: Milestone status as reported in the progression notes. Percentages are reported estimates, not independently measured test results.

Milestone	Completion criteria / current position	Reported completion
M1: Bug fixes and `localStorage` layer	JavaScript errors addressed and mock store made consistent.	100%
M2: Design system and styling	Core `style.css`, colours, and reusable components implemented; authentication and responsive styling remain.	80%
M3: HTML page migration	Page updates are ready for batch migration; most page-level migration remains.	15%
M4: Dynamic polish	CSS keyframes exist; observer, scroll listener, and toast work remain.	20%
M5: End-to-end QA	Full user-flow verification is dependent on the preceding UI and interaction work.	Pending
M6: Backend and MySQL integration	Express routes, schema, and persistent APIs replacing `localStorage`.	Pending
M7: Concurrency-safe joining	Transactional capacity enforcement verified by concurrency tests.	Planned
M8: Google OAuth and access control	Server-verified Google sign-in with institutional-domain restriction and route authorisation.	Planned
M9: Deployment and documentation	Production release verified; documentation updated.	Pending

10. Current Limitations and Dependencies

- **Local-only persistence:** application data currently use browser `localStorage` and mock data; they are not yet synchronised with a central MySQL database.
- **Authentication scope:** the current client-side authentication is a development simulation and should not be treated as production-grade identity verification. Google OAuth is planned but not implemented.
- **Eligibility verification:** restriction to eligible Thapar students depends on server-side institutional-domain validation (or another approved mechanism), which will be possible only after backend and OAuth integration.
- **Concurrency control:** team-capacity and duplicate-request rules are enforced only in the interface of a single browser; server-side transactional enforcement is pending.

- **Hackathon data integrity:** stable hackathon identifiers, validation, and deduplication rules have been proposed but not implemented.
- **UI migration incomplete:** page-level styling and responsive behaviour still require implementation and verification.
- **Testing pending:** the complete user workflow and edge cases have not yet been reported as fully verified.
- **Backend integration pending:** Express routes, database persistence, and server-side enforcement must be connected before the platform can be evaluated as a full-stack application.

11. Conclusion and Next Steps

The current development phase has stabilised the frontend's local data operations, repaired key interactions, improved the mock data, and established a reusable visual design system based on the Thapar crimson-and-white palette. Organising the work according to the standard Software Development Life Cycle has made the project's position clear: planning and requirements analysis are complete, design and implementation are in progress, and testing, deployment, and maintenance are still ahead. The SDLC also makes dependencies explicit, particularly that security and concurrency guarantees can only be verified once the backend exists.

The team plans to introduce Google OAuth 2.0 / OpenID Connect sign-in to simplify registration and login. Access will remain restricted to eligible Thapar students through server-side verification of the authenticated email against the approved institutional domain; Google sign-in on its own will not be treated as proof of eligibility.

Two technical challenges have been identified. Overlapping hackathon listings require stable hackathon identifiers, validation, and identifier-based filtering, while concurrent requests for a team's final position require authoritative backend capacity checks protected by database transactions, locking, and constraints. Both sets of fixes are proposed and have not yet been implemented.

The immediate next steps are to complete the page migrations and supporting CSS, implement and test the planned interactions, and verify the end-to-end workflow, including hackathon-listing integrity. The team will then connect the Node.js/Express backend to MySQL, replace the `localStorage` simulation with persistent APIs, implement concurrency-safe joining and Google OAuth, and carry out integration, security, and concurrency testing before deployment. Until then, the current version should be understood as a frontend simulation rather than a completed full-stack platform.